



OPTATIVA Machine Learning

Autor/a: Núria Pujol Vilanova | Curs: 2025-2026

Institut TIC Barcelona

Bucles for

Els bucles `for` són la manera més directa i llegible de recórrer col·leccions o seqüències d'elements en Python. A diferència d'altres llenguatges que treballen principalment amb índexs, Python permet iterar directament sobre l'objecte iterable (llista, cadena, tupla, diccionari, etc.).

El `for` extreu un element en cada iteració i l'assigna a una variable temporal per fer-ne ús.

```
In [ ]: colors = ["vermell", "verd", "blau", "groc"]
        for color in colors:
            print(color)
```

Utilitzar el nom en singular `color` quan iterem sobre `colors` és una bona pràctica de llegibilitat.

Si, per algun motiu, necessitem poder accedir o consultar el valor de l'índex de l'element actual del bucle podem utilitzar la funció `enumerate()`, que proporciona parells (índex, element):

```
In [ ]: for i, color_actual in enumerate(colors):
        print(f"El color {color_actual} es troba en la posició {i}")
```

D'aquesta manera podem accedir tant a la posició com al valor.

En el cas dels diccionaris, es poden recórrer per claus, valors o per parells.

```
In [ ]: paisos_i_capitals = {
        "Espanya": "Madrid",
        "França": "París",
        "Itàlia": "Roma",
        "Alemanya": "Berlín",
        "Portugal": "Lisboa"
    }

    for pais, capital in paisos_i_capitals.items():
        print(f"La capital de {pais} és {capital}.")

    for pais in paisos_i_capitals.keys():
        print(f"La capital de {pais}")
```

```
for capital in països_i_capitals.values():
    print(f"és {capital}.")
```

Bucles while

El bucle `while` repeteix un bloc de codi mentre es compleixi una determinada condició. Això el fa ideal per situacions on no sabem a priori quantes vegades heurem d'executar el nostre codi.

```
In [ ]: contador = 5
while contador > 0:
    print(contador)
    contador -= 1
```

També trobaràs que moltes vegades aquesta condició utilitza un objecte mutable (com una llista, exemple):

```
In [ ]: alumnos = ["Anna", "Marc", "Miquel"]

while alumnos:
    alumne = alumnos.pop()
    print(f"{alumne} ha sortit de l'aula")
```

L'operador walrus `:=` permet assignar i comparar en un sol pas, fent el codi més net.

```
In [ ]: while (entrada := input("Escriu 'sortir' per acabar: ").lower()) != 'sortir':
    print(f"Has escrit: {entrada}")
```

Sense utilitzar l'operador walrus (`:=`), cal inicialitzar la variable abans del bucle i tornar a demanar l'entrada dins.

La forma equivalent seria:

```
entrada = input("Escriu 'sortir' per acabar: ").lower()
while entrada != 'sortir':
    print(f"Has escrit: {entrada}")
    entrada = input("Escriu 'sortir' per acabar: ").lower()
```

Modificadors del control del bucle

Aquestes instruccions permeten modificar el comportament normal del bucle sense necessitat d'utilitzar variables externes o estructures addicionals, fent que el codi sigui més clar i pythònic.

- `break` : serveix per interrompre immediatament l'execució del bucle. S'utilitza per aturar la iteració, per exemple, quan s'ha trobat l'element que es buscava.

```
for n in range(10):
    if n == 5: #finalitza el bucle per n = 5
        break
    print(n)
```

- `continue` : fa que la iteració actual finalitzi i salti a la següent.

```
for n in range(6):
    if n % 2 == 0:
        continue # salta els parells
    print(n)
```

- `else` després de `for` o `while` : ús poc habitual però molt útil per detectar si un bucle ha acabat "naturalment".

```
nombres = [1, 3, 5, 9]
objetiu = 7
```

```
for nombre in nombres:
    if nombre == objetiu:
        print("Trobat!")
        break
else:
    print("No trobat")
```

Do-While (com emular-lo)

Python no té l'estructura de bucle do-while (típica d'altres llenguatges) però en podem emular el seu comportament.

Recordem que en el bucle `do-while`, el bucle s'executa com a mínim una vegada perquè la condició de sortida es comprova al final de la iteració, a diferència del bucle `while`, que comprova la condició abans d'executar el bucle i podria no executar-se mai si la condició de sortida ja es compleix.

La manera més habitual d'emular un `do-while` és utilitzant `while True + break` (un bucle infinit amb una sentència `break` per sortir-ne).

```
In [ ]: from random import randint

num_secret = randint(0, 10)

while True:
    num_entrat = int(input(f"Endevina el número entre 0 i 10 "))
    if num_entrat == num_secret:
        break
    print("No l'has encertat.")

print(f"Ho has encertat! El número secret era {num_secret}")
```

Aquest patró garanteix que el cos del bucle s'executi almenys una vegada, ja que la condició d'aturada (`if num_entrat == num_secret`) es comprova al final del cicle mitjançant un `break`.

Iteradors i generadors

Un **iterable** és qualsevol objecte sobre el qual es pot iterar, com llistes, cadenes i diccionaris. Un iterador és un objecte que manté l'estat de la iteració, proporcionant elements un a un amb `next()`.

Els bucles `for` fan servir internament iteradors.

```
In [ ]: fruites = ['poma', 'plàtan', 'taronger']
iterador = iter(fruites) # Creem l'iterador de la llista

print(next(iterador)) # 'poma'
print(next(iterador)) # 'plàtan'
print(next(iterador)) # 'taronger'
# Si tornem a cridar next(iterador), es llençarà StopIteration
```

```
In [ ]: print(next(iterador)) # Obtenim l'error StopIteration
```

Els **generators** són funcions especials que, en lloc de retornar tota la seqüència d'una vegada amb `return`, "serveixen" els valors un a un amb `yield`.

Això els fa molt eficients per a grans quantitats de dades o fins i tot seqüències infinites.

```
In [ ]: def generador_parells(n):
        for i in range(n + 1):
            if i % 2 == 0:
                yield i

for parell in generador_parells(10):
    print(parell, end = ' ')
```

```
In [ ]: g = generador_parells(10)
print(next(g)) # 0
print(next(g)) # 2
print(next(g)) # 4
print(next(g)) # 6
print(next(g)) # 8
print(next(g)) # 10
```

```
In [ ]: print(next(g)) # Error
```

Aquest procés és el que fa Python internament quan fem un bucle `for`.

El `range()` no és una llista, sinó un objecte generator, que funciona de manera similar als generators.

```
In [ ]: for y in range(10):
        print(y, end=" ")
```

```
In [ ]: print(range(10))
```

```
In [ ]: type(range(10))
```

```
In [ ]: list(range(10))
```

Compressions de llistes (*List Comprehension*)

Les *list comprehension* són una característica molt potent i pythonica que permet crear noves llistes de forma compacta, clara i eficient. La seva idea és poder escriure en una sola línia el procés de transformació o filtratge dels elements d'una col·lecció existent per crear-ne una de nova.

Aquesta seria la seva sintaxis bàsica:

```
[nova_expressio for element in iterable if condicio]
```

Veient-me un exemple, que potser serà més fàcil d'entendre:

```
In [ ]: nombres = [1, 2, 3, 4, 5]
        quadrats = [x**2 for x in nombres]
        print(quadrats) # [1, 4, 9, 16, 25]
```

Exemple de filtratge:

```
In [ ]: # nombres declarada anteriorment
        parells = [x for x in nombres if x % 2 == 0]
        print(parells) # [2, 4]

        parells = [x for x in range(10) if x % 2 == 0]
        print(parells) # [0, 2, 4, 6, 8]

        estat_paritat = ['Parell' if x % 2 == 0 else 'Senar' for x in nombres]
        print(estat_paritat) # ['Senar', 'Parell', 'Senar', 'Parell', 'Senar']

        llista = [x for x in range(100) if x % 2 == 0 if x % 5 == 0]
        print(llibra) # [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
```

Funció any()

La funció any() de Python serveix per comprovar si almenys un element d'un iterable és cert (*truthy*). Retorna:

- **True** si almenys un element és veritable (retorna **True** quan es converteix en booleà).
- **False** si tots els elements són falsos o està buit (retorna **False** quan es converteix a booleà).

any() recorre l'iterable i comprova cada element: si en troba un que sigui veritable, atura la comprovació i retorna **True** immediatament. Si no troba cap element veritable després de recórrer tots, retorna **False**.

```
bool(1) # True
bool(0) # False
bool([]) # False
bool('a') # True
bool('') # False
```

Aquest tipus de funcionament s'anomena avaluació vaga/mandrosa (*lazy evaluation*) i fa que `any()` sigui molt eficient per comprovar condicions dins de col·leccions grans.

Comprova si hi ha algun element que no sigui zero o `False` :

```
In [ ]: llista1 = [0,0,0,0]
        llista2 = [0,0,0,5]
        llista3 = [False, False, False]
        llista4 = [False, False, True]

        print(any(llista1)) #False
        print(any(llista2)) #True
        print(any(llista3)) #False
        print(any(llista4)) #True
```

Validar que un formulari té algun valor no buit amb `any()` o tots els camps emplenats amb `not any()` :

```
In [ ]: entrades = ['Joan', 'joan@example.com', 'Barcelona', '']

        if any(entrada for entrada in entrades):
            print("Com a mínim hi ha un dels camps emplenat")
        else:
            print("Hi ha tots els camps buits del formulari")

        if not any(entrada for entrada in entrades):
            print("Totes les entrades estan correctament emplenades.")
        else:
            print("Hi ha camps buits en el formulari")
```

Buscar una paraula concreta en una llista de frases o missatges:

```
In [ ]: missatges = [
        "TypeError: can only concatenate str (not 'int') to str",
        "ValueError: invalid literal for int() with base 10",
        "Runtime successfully completed"
    ]

        hi_ha_error = any('Error' in missatge for missatge in missatges)

        if hi_ha_error:
            print("El programa ha finalitzat per algun error.")
        else:
            print("El programa ha finalitzat satisfactòriament sense errors.")
```

També disposem de la funció `all()`, similar a `any()`, però amb un funcionament complementari.

Mentre que `any(iterable)` retorna `True` si almenys un element de l'iterable és veritable (*truthy*), `all(iterable)` retorna `True` només si tots els elements són veritables.

```
In [ ]: notes = [6, 7, 5, 4, 8]

        # Comprova si l'estudiant ha aprovat totes les assignatures (totes les notes >=
```

```
ha_aprobat_totes = all(nota >= 5 for nota in notes)

# Comprova si l'estudiant ha aprovat almenys una assignatura (almenys una nota >
ha_aprobat_almenys_una = any(nota >= 5 for nota in notes)

print("Ha aprovat totes les assignatures?", ha_aprobat_totes)
print("Ha aprovat almenys una assignatura?", ha_aprobat_almenys_una)
```